

Documentación IEEE830

1. Introducción

El presente documento esta basado en el estándar de Especificación de Requisitos Software (ERS) del sistema desarrollado como Proyecto Final del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web (DAW).

Este documento ha sido elaborado siguiendo la estructura recomendada por mi tutor de proyecto Arturo que es el estándar IEEE 830-1998, con el objetivo de definir de forma mas clara posible y completa todos aquellos requisitos funcionales y no funcionales del sistema el cual voy a desarrollar como proyecto final de grado.

1.1 Propósito

El propósito de este documento es especificar detalladamente los requisitos y funcionales del sistema que voy a desarrollar. Que en este caso va ser un aplicación web full-stack ya que tendrá un aplicación backend donde se desarrollara una API REST para así poder servir los datos a la segunda aplicación que va ser un Rest Client que consumirá la API que desarrollare previamente. La aplicacion se llamara previamente **BookSocial**

Este documento esta dirigido a:

- El desarrollador del sistema.
- El tribunal de profesores del proyecto.
- Mi tutor de practicas
- Cualquier persona interesada en comprender el alcance, funcionamiento y limitaciones del sistema.

La ERS servirá como referencia para:

- El diseño del sistema.
- La implementación.
- La planificación de pruebas.
- La validación final del proyecto.

1.2 Ámbito del Sistema

Nombre del sistema

BookSocial - Plataforma Web de Venta y Comunidad Literaria

Descripción general

El proyecto consiste en el desarrollo de una **aplicación web de venta de libros** que combina funcionalidades de **e-commerce** con un **componente social** orientado a fomentar la lectura y la interacción entre usuarios. Esta aplicación estará basada en el desarrollo de una API REST con Spring boot la cual será consumida por una aplicación cliente que en este caso primero será un prototipo con Thymeleaf + Bootstrap.

La aplicación permitirá la compra de **libros, mangas y cómics**, así como la consulta y participación en un sistema de valoraciones, opiniones y seguimiento de obras y autores. Los usuarios podrán descubrir contenido mediante distintos filtros y rankings, inspirados en plataformas como MyAnimeList, FilmAffinity o Letterboxd. Los usuarios también podrán seguir a otros usuarios para poder ver sus listas de favoritos, las opiniones que ha compartido para poder fomentar como ese aspecto social que le queremos dar a la aplicación.

Además se añadirá una funcionalidad para poder tener un **tipo de suscripción mensual** para los usuarios la cuales les ofrecerá cierto tipo de beneficios a los usuarios que las tengan como puede tener rebajas en los en los productos que se venden, que se puedan apuntar a eventos que se hagan en la web y otras cosas que podemos seguir desarrollando.

Además, existirán **usuarios administradores** encargados de gestionar determinadas partes del sistema, como el catálogo o el contenido publicado. Los administradores también podrán borrar comentarios que no sean oportunos gestionar y crear los eventos para aquellos usuarios que estén suscritos.

Funcionalidades principales del sistema

El sistema permitirá:

- Registro e inicio de sesión de usuarios.
- Gestión de roles (invitado, registrado, suscrito y administrador).
- Visualización y filtrado de catálogo de obras.
- Gestión de pedidos y estados.
- Sistema de comentarios jerárquicos.
- Sistema de likes.
- Seguimiento de obras y usuarios.
- Gestión de múltiples ediciones por obra.
- Sistema de suscripción mensual con renovación automática.
- Panel de administración para gestión global del sistema.

Que no hará el sistema

El sistema no contemplará en esta versión:

- Integración con pasarelas de pago reales.
- Gestión logística real de envíos.
- Aplicación móvil nativa.
- Funcionamiento sin conexión.
- Microservicios distribuidos.

Objetivos y beneficios

El sistema pretende:

- Centralizar en una sola plataforma la compra y valoración de obras literarias.
- Fomentar la interacción social entre lectores.
- Simular un entorno profesional real basado en arquitectura REST.
- Aplicar buenas prácticas de diseño, seguridad y estabilidad.

1.3 Definiciones, Acrónimos y Abreviaturas

Término	Definición
ERS	Espeficacion de Requisitos Software
API REST	Interfaz de comunicación basada en arquitectura REST
CRUD	Operaciones de creación, lectura, actualización y eliminación
JWT	JSON Web Token para autenticación
Obra	Entidad base que representa libro, manga o cómic
Edición	Versión específica de una obra asociada a una editorial
Stock	Cantidad disponible de una edición
Suscripción	Plan mensual con beneficios exclusivos
Usuario Invitado	Usuario sin autenticación
Usuario Registrado	Usuario autenticado
Usuario Suscrito	Usuario registrado con plan activo
Administrador	Usuario con privilegios de gestión

1.4 Referencias

Referencia	Link
IEEE Std 830-1998	https://www.fdi.ucm.es/profesor/gmendez/docs/is0809/ieee830.pdf
Documentación oficial de Spring Boot	https://spring.io/projects/spring-boot
Libro: Desarrollo web en entorno servidor. Spring Framework	https://www.garceta.es/catalogo/libro.php?ISBN=978-84-1903-446-5&idd=12

1.5 Visión General del Documento

El presente documento se estructura de la siguiente manera:

- **Sección 1:** Introducción.
- **Sección 2:** Descripción general del sistema y su contexto.
- **Sección 3:** Requisitos específicos, incluyendo requisitos funcionales y no funcionales.
- **Sección 4:** Apéndices.

La **Sección 2** describirá el contexto del producto y su relación con el entorno.

La **Sección 3** detallará los requisitos de forma estructura y verificable

2. Descripción del Producto

2.1 Perspectiva del Producto

El cliente BookSocial es una aplicación web full-stack basada en arquitectura cliente-servidor.

El producto no forma parte de un sistema empresarial mayor. Se trata de un sistema independiente desarrollado como Proyecto Final del Ciclo Formativo de Grado Superior en Desarrollo en Desarrollo de Aplicaciones Web.

Arquitectura general

El sistema estará compuesto por:

- Backend desarrollado con Spring Boot 4.3
- API REST como mecanismo de comunicación.
- Cliente web (prototipo inicial con Thymeleaf + Bootstrap).
- Base de datos relacional en producción.
- Base de datos H2 en desarrollo.
- Despliegue en entorno Docker.

El frontend consumirá los datos exclusivamente a través de la API REST proporcionada por el backend.

Relación con otros sistemas

El sistema no dependerá de plataformas externas para su funcionamiento principal. No se integrará en esta versión con:

- Pasarelas de pago reales.
- Servicios externos de autenticación.
- Sistemas logísticos de envío.

Podrá utilizar herramientas externas como:

- Git para control de versiones.
- Docker para despliegue.
- Base de datos MySQL en producción.
- H2 en entorno de desarrollo

El sistema funcionará exclusivamente en entorno online.

2.2 Funciones del Producto

El sistema proporcionará, a grandes rasgos, las siguientes funciones principales:

Gestión de usuarios

- Registros e inicio de sesión.
- Gestión de perfiles.
- Gestión de roles (invitado, registrado, suscrito, administrador).
- Administración de usuarios.

Gestión de catálogo

- Visualización de obras(libros, mangas y cómics).
- Gestión de ediciones por editorial.
- Filtrado por género, autor, año, valoración y popularidad.
- Administración del catálogo por parte del administrador.

Sistema de compra

- Gestión de carrito de compra.
- Control de stock por edición.
- Gestión de pedidos y estados.

- Historial de compras del usuario.

Sistema social

- Publicación de comentarios jerárquicos.
- Edición y eliminación de comentarios propios.
- Sistema de likes.
- Seguimiento de obras y usuarios.

Sistema de suscripción

- Suscripción mensual con renovación automática.
- Aplicación de descuentos.
- Acceso a eventos exclusivos.
- Identificación visual mediante badge.
- Acceso a estadísticas avanzadas.

Panel de administración

- Gestión de catálogo.
- Moderación de contenido.
- Gestión de suscripciones.
- Supervisión general del sistema.

2.3 Características de los Usuarios

El sistema contempla los siguientes perfiles de usuario:

Usuario Invitado

- No requiere conocimientos técnicos.
- Puede consultar el catálogo.
- No puede interactuar socialmente ni realizar compras.

Usuario Registrado

- Nivel básico de conocimientos digitales.
- Familiarizado con plataformas de e-commerce.
- Puede realizar compras e interactuar socialmente.

Usuario Suscrito

- Usuario registrado con plan activo.
- Accede a funcionalidades avanzadas.

- No requiere conocimientos técnicos adicionales.

Usuario Administrador

- Usuario con conocimientos técnicos medios.
- Responsable de la gestión y supervisión del sistema.
- Accede a paneles de control y estadísticas.****

2.4 Restricciones

El desarrollo del sistema estará condicionado por las siguientes restricciones:

Tecnológicas

- Backend obligatorio en Java con Spring Boot.
- Uso de API REST.
- Persistencia mediante JPA/Hibernate.
- Base de datos relacional (H2 en desarrollo, MySQL en producción)
- Uso de JWT para autenticación.

Académicas

- Cumplimiento del estándar IEEE 830
- Entregas por hitos establecidos en planificación.
- Documentación completa del código y API.

Seguridad

- Control de acceso basado en roles.
- Autenticación mediante login y token.
- Restricción de acciones según permisos.

Hardware

- El sistema deberá funcionar en equipos estándar con navegador moderno.
- No se optimiza para dispositivos móviles nativos.

2.5 Suposiciones y Dependencias

El sistema se basa en las siguientes suposiciones:

- El usuario dispondrá de conexión a internet.
- El servidor estará operativo de forma continua.
- La base de datos estará correctamente configurada.
- No se requerirá integración con sistemas externos reales.

Dependencias técnicas:

- Spring Boot.
- Base de datos relacional.
- Navegador web compatible.
- Docker para despliegue (fase final).

Si cualquiera de estas condiciones cambia, podrían modificarse los requisitos del sistema.

2.6 Requisitos Futuros

Se contemplan posibles ampliaciones futuras:

- Integración con pasarelas de pago reales.
- Sistema de notificaciones en tiempo real.
- Aplicación móvil nativa.
- Exportación de datos en PDF o CSV.
- Sistema avanzado de estadísticas y recomendaciones.
- Escalado hacia arquitectura de microservicios.

Estas mejoras no forman parte del alcance actual, pero podrían implementarse en futuras versiones.

3. REQUISITOS ESPECIFICOS

3.1 Interfaces Externas

3.1.1 Interfaz de Usuario

- **IU-01:** El sistema proporcionará una interfaz web accesible desde cualquier navegador moderno.
- **IU-02:** La interfaz permitirá una navegación mediante menús estructurados.
- **IU-03:** La interfaz mostrará mensajes de error claros en caso de operación inválida.
- **IU-04:** El sistema diferenciará visualmente los roles de usuario.
- **IU-05:** El usuario suscrito mostrará un badge identificativo en su perfil.

3.1.2 Interfaces Software

- **IS-01:** EL frontend consumirá datos exclusivamente mediante API REST
- **IS-02:** La autenticación se realizará mediante JWT.
- **IS-03:** El sistema utilizará base de datos relacional MySQL o Postgress.
- **IS-04:** Se utilizará JPA/Hibernate para persistencia.

3.1.3 Interfaces de Comunicación

- **IC-01:** La comunicación cliente-servidor se realizará mediante HTTP.
- **IC-02:** Los datos se intercambiarán en formato JSON.
- **IC-03:** El sistema requerirá conexión a internet.

3.2 Funciones

El tipo de organización recomendada por el estándar IEEE830 que he escogido para redactar este punto es la forma organizativa de **Por Tipos de usuario**, el motivo el cual he escogido esta forma es por que mi sistema se base en roles claramente diferenciados, lo que facilita trazabilidad y diseño.

3.2.1 Requisitos del Usuario Invitado

- RF-INV-01: El sistema permitirá visualizar el catálogo de obras.
- RF-INV-02: Permitirá filtrar por género, autor, año y tipo.
- RF-INV-03: Permitirá visualizar detalle de obra.
- RF-INV-04: No permitirá realizar compras.
- RF-INV-05: No permitirá publicar comentarios.

3.2.2 Requisitos del Usuario Registrado

Autenticación

- RF-REG-01: El sistema permitirá registro mediante formulario.
- RF-REG-02: Permitirá inicio de sesión.
- RF-REG-03: Permitirá cierre de sesión.

Compra

- RF-REG-04: Permitirá añadir productos al carrito.
 - RF-REG-05: Permitirá modificar cantidades.
 - RF-REG-06: Permitirá eliminar productos del carrito.
 - RF-REG-07: El carrito permanecerá asociado al usuario autenticado.
 - RF-REG-08: El sistema validará stock antes de confirmar pedido.
 - RF-REG-09: Permitirá comprar múltiples unidades.
 - RF-REG-10: Permitirá cancelar pedidos en estado Pendiente.
 - RF-REG-11: El sistema gestionará estados: Pendiente, Confirmado, Cancelado.
-

Sistema Social

- RF-REG-12: Permitirá publicar comentarios sobre obras y autores.
 - RF-REG-13: Permitirá responder a comentarios.
 - RF-REG-14: Permitirá editar comentarios propios.
 - RF-REG-15: Permitirá eliminar comentarios propios.
 - RF-REG-16: Permitirá dar un único like por comentario.
 - RF-REG-17: Permitirá retirar su propio like.
 - RF-REG-18: Permitirá seguir obras.
 - RF-REG-19: Permitirá seguir usuarios.
-

Perfil

- RF-REG-20: Permitirá visualizar historial de compras.
 - RF-REG-21: Permitirá visualizar opiniones publicadas.
 - RF-REG-22: Permitirá editar datos personales.
-

3.2.3 Requisitos del Usuario Suscrito

- RF-SUS-01: El sistema aplicará descuento fijo en compras.
- RF-SUS-02: Permitirá acceso a eventos exclusivos.
- RF-SUS-03: Permitirá acceso anticipado a determinadas obras.
- RF-SUS-04: Permitirá visualizar estadísticas avanzadas.
- RF-SUS-05: Permitirá cancelar suscripción en cualquier momento.
- RF-SUS-06: La suscripción se renovará automáticamente cada 30 días.

3.2.4 Requisitos del Administrador

- RF-ADM-01: Gestionará catálogo completo (CRUD).
 - RF-ADM-02: Gestionará autores.
 - RF-ADM-03: Gestionará editoriales.
 - RF-ADM-04: Gestionará stock por edición.
 - RF-ADM-05: Moderará comentarios.
 - RF-ADM-06: Gestionará usuarios.
 - RF-ADM-07: Podrá modificar o cancelar suscripciones.
 - RF-ADM-08: Podrá visualizar estadísticas globales.
 - RF-ADM-09: Podrá gestionar eventos exclusivos.
-

3.3 Requisitos de Rendimiento

- RR-01: El sistema soportará al menos 50 usuarios simultáneos.
 - RR-02: El tiempo de respuesta promedio será inferior a 3 segundos.
 - RR-03: El sistema podrá almacenar miles de registros de obras.
 - RR-04: El sistema deberá mantener integridad de stock en concurrencia.
-

3.4 Restricciones de Diseño

- RD-01: Backend desarrollado en Spring Boot.
 - RD-02: Uso obligatorio de API REST.
 - RD-03: Base de datos relacional.
 - RD-04: Seguridad implementada mediante Spring Security.
 - RD-05: Control de acceso basado en roles.
-

3.5 Atributos del Sistema

Seguridad

- AS-SEG-01: Solo usuarios autenticados podrán comprar.
- AS-SEG-02: Solo administrador podrá gestionar catálogo.

- AS-SEG-03: Los tokens JWT deberán validarse en cada petición protegida.

Fiabilidad

- AS-FIAB-01: El sistema mantendrá consistencia de datos.
- AS-FIAB-02: El sistema controlará errores mediante excepciones.

Mantenibilidad

- AS-MAN-01: El sistema seguirá arquitectura en capas.
- AS-MAN-02: El código incluirá documentación Javadoc.

Portabilidad

- AS-POR-01: El sistema podrá desplegarse mediante Docker.
-

3.6 Otros Requisitos

- OR-01: El sistema funcionará exclusivamente online.
- OR-02: No se integrará con pasarelas de pago reales en esta versión.
- OR-03: No incluirá aplicación móvil nativa.
- OR-04: No funcionará sin conexión.

3.7 Modelo de Casos de Uso del Sistema

Con el objetivo de representar de forma gráfica las funcionalidades del sistema BookSocial y la interacción de los distintos tipos de usuarios con la aplicación, se ha elaborado un diagrama de casos de uso.

Este diagrama permite visualizar las operaciones que los usuarios pueden realizar dentro del sistema y la relación entre los distintos módulos funcionales de la aplicación.

- Usuario invitado
- Usuario registrado
- Usuario suscrito
- Administrador

Cada uno de estos actores posee diferentes niveles de acceso y funcionalidades dentro del sistema.

3.7.1 Actores del Sistema

Usuario Invitado:

El usuario invitado es aquel que accede a la plataforma sin haberse autenticado.

Este tipo de usuario dispone únicamente de funcionalidades de consulta, como la visualización del catálogo de obras y el acceso a la información básica de cada obra.

Entre sus acciones disponibles se encuentran:

- Visualizar el catálogo de obras.
- Filtrar obras según diferentes criterios.
- Consultar el detalle de una obra.

Usuario Registrado:

El usuario registrado es aquel que ha creado una cuenta dentro de la plataforma y ha iniciado sesión en el sistema.

Este usuario dispone de todas las funcionalidades del usuario invitado, además de funcionalidades sociales y de compra.

Entre sus acciones principales se encuentran:

- Gestionar su carrito de compra.
- Confirmar pedidos.
- Cancelar pedidos.
- Publicar comentarios sobre obras.
- Editar y eliminar sus propias comentarios
- Responder a comentarios de otros usuarios
- Dar o quitar "like" a comentarios
- Seguir obras
- Seguir a otros usuarios

Estas funcionalidades permiten fomentar la interacción social dentro de la plataforma.

Usuario Suscrito:

El usuario suscrito es un usuario registrado que dispone de una suscripción activa dentro de la plataforma.

El tipo de usuario puede acceder a funcionalidades adicionales relacionadas con su plan de suscripción.

Entre estas funcionalidades se incluyen:

- Suscribirse a la plataforma
- Cancelar sus suscripción
- Acceder a estadísticas avanzadas
- Participar en eventos exclusivos organizados por la plataforma.

Estas funcionalidades permiten ofrecer ventajas adicionales a los usuarios más activos de la comunidad.

Administrador:

El administrador es el usuario encargado de gestionar y supervisar el correcto funcionamiento de la plataforma.

Dispone de privilegios avanzados que le permiten administrar distintos elementos del sistema.

Entre sus principales responsabilidades se encuentran:

- Gestionar el catálogo de obras
- Gestionar autores y editoriales
- Controlar el stock de las ediciones
- Moderar comentarios publicados por los usuarios
- Gestionar usuarios registrados
- Gestionar suscriptores
- Gestionar eventos organizados dentro de la plataforma

El administrador actúa como figura de control para garantizar la calidad del contenido y el correcto funcionamiento del sistema.

3.7.2 Organización funcional del sistema

El sistema se organiza en diferentes subistemas funcionales que agrupan los distintos casos de uso:

Gestion de Catálogo:

Permite consultar las obras disponibles dentro de la plataforma.

Incluye las siguientes funcionalidades:

- Visualizar catálogo
- Filtrar obras
- Ver detalle de obra

Sistema de Autenticación:

Gestiona el acceso de los usuarios a la plataforma mediante un sistema de registro e inicio de sesión.

Incluye las siguientes funcionalidades:

- Publicar comentarios
- Editar comentarios
- Eliminar comentarios
- Responder a comentarios
- Dar "like"
- Seguir obras
- Seguir usuarios

Sistema de Compra:

Gestiona el proceso de compra de obras dentro de la plataforma.

Incluye las siguientes funcionalidades:

- Gestionar carrito
- Confirmar pedido
- Cancelar pedido

Sistema de Suscripción

Permite gestionar el plan de suscripción mensual de los usuarios.

Incluye funcionalidades como:

- Suscribirse
- Cancelar suscripción
- Acceder a estadísticas avanzadas

Administración del sistema:

Este módulo agrupa las funcionalidades disponibles exclusivamente para los administradores.

Incluye la gestión de:

- Catálogo de obras
- Autores
- Editoriales
- Stock
- Comentarios

- Usuarios
- Suscriptores
- Eventos

3.8 Modelo de Dominio y Estructura de Datos del Sistema

Con el objetivo de definir la estructura interna del sistema BookSocial, se ha elaborado un modelo de Domino basado en un diagrama de clases que representa las principales entidades del sistema, sus atributos y las relaciones existentes entre ellas.

El modelo conceptual permite representar de forma estructurada los elementos fundamentales de la la aplicación y sirve como base para el posterior diseño de la base de datos y la implementación del sistema mediante entidades JPA en el bakcend desarrollado con Spring Boot.

El modelo de clases se organiza en cuatro grandes bloques funcionales que reflejan las principales áreas de la aplicación: **gestión de usuarios, gestión de obras, sistema social y sistema de compra.**

3.8.1 Gestión de Usuarios

La clase Usuario contiene información básica como el identificar del usuario, nombre del usuario, correo electrónico, contraseña, fecha de registro y estado de actividad dentro del sistema, Además, cada usuario dispone de un rol que determina los permisos disponibles dentro de la plataforma.

Los roles del sistema se representan mediante el enumerado **Role**, que distingue entre los siguientes tipos de usuarios:

- Usuario Invitado
- Usuario registrado
- Usuario suscrito
- Usuario administrador

Esta estrategia permite gestionar la autorización del sistema mediante mecanismos de seguridad proporcionados por Spring Security.

El sistema incluye además la entidad **Suscripción**, que representa el plan mensual al que pueda acceder un usuario registrado. Esta relación es opcional, ya que no todos los usuarios disponen de una suscripción activa. La suscripción permite acceder a beneficios adicionales dentro de la plataforma, como descuentos o acceso a eventos exclusivos.

También se incorpora la entidad **Evento**, que representa actividades organizadas dentro de la plataforma para los usuarios y obras dentro de la plataforma. De esta manera, los usuarios

pueden seguir tanto a otros usuarios como a determinadas obras para recibir información sobre su actividad o novedades.

3.8.2 Gestión de Obras

La entidad central del sistema es **Obra**, que representa cualquier elemento del catálogo disponible en la plataforma, incluyendo libros mangas y cómics.

La clase Obra contiene atributos comunes como:

- título
- descripción
- fecha de publicación
- género
- imagen
- valoración media

Para diferenciar los distintos tipos de obra se utiliza el enumerador **Tipo**, que permite identificar si una obra corresponde a un libro, manga o cómic.

Cada obra puede estar asociado a uno o varios **Autores**, estableciendo una relación de muchos a muchos entre ambas entidades. Esto permite modelar correctamente casos en los que una obra tiene varios autores o un autor participa en múltiples obras.

Las obras pueden tener múltiples **Ediciones**, que representan versiones específicas de una obra asociada a una editorial concreta. Cada edición incluye información como el ISBN, el precio, la fecha de edición y el stock disponible.

La entidad **Editorial** representa las distintas editoriales que publican las obras disponibles en el catálogo.

En el caso de los **Mangas**, las obras se organizan mediante capítulos agrupados en tomos. Cada tomo pertenece a una edición concreta de una obra, y cada tomo contiene varios capítulos.

Este modelo permite representar correctamente la estructura editorial habitual de los mangas, donde una editorial puede agrupar los capítulos en tomos de distinta manera.

Para los **cómics**, la organización se realiza mediante **volúmenes o números**, que también pertenecen a una edición de la obra.

3.8.3 Sistema Social

El sistema incorpora funcionalidades sociales que permiten a los usuarios interactuar entre sí mediante comentarios y valoraciones.

La entidad **Comentario** permite que los usuarios publiquen opiniones sobre las obras disponibles en la plataforma. Cada comentario está asociado a un usuario y una obra concreta.

Además, el sistema implementa un modelo jerárquico de comentarios mediante una autor relación dentro de la propia entidad Comentario. Esto permite que los usuarios puedan responden a comentarios existentes, generando conversaciones dentro de cada obra.

Para fomentar la interacción entre usuarios, el sistema incorpora la entidad **Like**, que permite a los usuarios indicar su aprobación sobre comentarios publicados por otros usuarios. Cada usuarios puede dar un único "like" por comentario, garantizando la integridad de la interacción.

3.8.4 Sistema de Compra

EL sistema de compra se basa en un modelo clásico de pedidos utilizado en aplicaciones de comercio electrónico.

La entidad **Pedido** representa una compra realizada por un usuario. Cada pedido incluye información como la fecha de creación, el estado de pedido y el importe total.

Los estados del pedido se gestionan mediante el enumerado **EstadoPedido**, que permite identificar el estado actual del proceso de compra:

- Pendiente
- Confirmado
- Cancelado
- Enviado
- Entregado

Cada pedido esta compuesto por varias **Líneas de Pedido**, representadas mediante la entidad **LineaPedido**. Cada línea de pedido almacena la edición adquirida, la cantidad de unidades y el precio unitario en el momento de la compra.

Este diseño permite mantener la coherencia histórica de los pedidos incluso si el precio de una edición cambia en el futuro.

3.8.5 Justificación del Modelo

El modelo de clases ha sido diseñado con el objetivo de mantener una estructura modular, escalable y coherente con los requisitos funcionales definidos en la especificación del sistema.

Se ha optado por evitar el uso de herencia en las entidades principales y utilizar en su lugar enumeraciones y relaciones entre clases. Esta decisión facilita la implementación en entornos

basados en JPA y reduce la complejidad del modelo.

Asimismo, la separación entre obra, edición y stock permite representar correctamente diferentes versiones de una misma obra publicada por distintas editoriales, garantizando un mayor flexibilidad en la gestión del catálogo.

 Diagrama de clases

3.10 Modelo de Datos del Sistema

3.10.1 Modelo Entidad- Relación

El modelo de datos del sistema BookSocial se ha diseñado utilizando un modelo entidad-relación con el que objetivo de representar de forma estructurada la información gestionada por la aplicación.

Este modelo define las entidades principales del sistema, sus atributos y las relaciones existentes entre ellas. La estructura de datos permite soportar las funcionalidades principales de la plataforma, incluyendo la gestión de usuarios, el catálogo de obras y facilitar la escalabilidad del sistema en futuras ampliaciones.

El sistema se organiza en cuatro bloques funcionales principales:

FOTO AQUÍ

3.10.2 Gestión de Usuarios

La entidad Usuario representa a las personas que interactúan con la plataforma. Cada usuario dispone de información básica como su nombre de usuario, correo electrónico, contraseña, rol dentro del sistema, fecha de registro y estado de actividad.

Los usuarios pueden realizar múltiples acciones dentro del sistema, como realizar pedidos, publicar comentarios o interactuar con otros usuarios.

Además, el sistema permite que los usuarios dispongan de una **suscripción mensual**, representada mediante la entidad **Suscripción**, la cual se relaciona con el usuario mediante una relación uno a uno opcional. Esto significa que no todos los usuarios tienen necesariamente una suscripción activa.

El sistema también incluye la entidad **Evento**, que representa actividades organizadas dentro de la plataforma. Los usuarios pueden inscribirse en eventos mediante una relación muchos a muchos gestionada por la tabla intermedia **usuario_evento**.

Otra funcionalidad importante es el sistema de **seguimiento**, representado mediante la entidad **Seguimiento**, que permite a los usuarios seguir tanto obras como otros usuarios dentro de la

plataforma.

3.10.3 Gestión del Catálogo de Obras

La entidad central del catálogo es **Obra**, que representa los distintos productos literarios disponibles en la plataforma, incluyendo libros mangas y cómics.

Cada obra contiene información general como títulos, descripción, fecha de publicación, género, imagen representativa y valoración media de los usuarios.

Una obra puede estar asociada a uno o varios **Autores**, estableciendo una relación muchos a muchos entre ambas entidades. Esta relación se gestiona mediante la tabla intermedia **obra_autor**, lo que permite representar correctamente casos en los que una obra tiene múltiples autores o un autor participa en varias obras.

Las obras pueden tener diferentes **Ediciones**, que representan versiones concretas de una misma obra publicadas por distintas editoriales. Cada edición incluye atributos como ISBN, precio, stock disponible y fecha de edición.

La entidad **Editorial** representa las empresas responsables de la publicación de las obras.

3.10.4 Estructura de Obras Complejas

El sistema contempla diferentes estructuras internas para representar correctamente la organización editorial de mangas y cómics.

En el caso de los **mangas**, las obras se organizan mediante **capítulos en tomos**. Cada tomo pertenece a una edición específica de una obra, y cada tomo contiene varios capítulos. Esta estructura permite representar correctamente el formato habitual de publicación de los mangas.

Para los **cómics**, el sistema utiliza la entidad **Volumen**, que representa los distintos números o volúmenes que componen una colección de cómics. Cada volumen pertenece a una edición concreta de la obra.

Este modelo permite gestionar diferentes estructuras editoriales sin necesidad de utilizar herencia en modelo de datos, lo que facilita la implementación mediante tecnologías de persistencia como JPA.

3.10.5 Sistema Social

El sistema incorpora funcionalidades sociales que permiten a los usuarios interactuar entre sí mediante comentarios y valoraciones.

La entidad **Comentario** representa las opiniones publicadas por los usuarios sobre una obra concreta. Cada comentario está asociado tanto a un usuario como a una obra.

Además, el sistema permite responder a comentarios mediante una **autorrelación dentro de la entidad Comentario**, lo que permite generar conversaciones jerárquicas entre usuarios.

Para fomentar la interacción social, se ha incorporado la entidad **Like**, que permite a los usuarios indicar su aprobación sobre comentarios publicados por otros usuarios. Cada like está asociado a un usuario y a un comentario concreto.

3.10.6 Sistema de Compra

El sistema de compra se basa en un modelo clásico de comercio electrónico.

La entidad **Pedido** representa una compra realizada por un usuario. Cada pedido incluye información como la fecha de creación, el estado del pedido y el importe total de la compra.

Los posibles estados del pedido se gestionan mediante la entidad **estado_pedido_enum**, que permite controlar el estado del proceso de compra.

Cada pedido puede contener múltiples productos, representados mediante la entidad **LineaPedido**. Cada línea de pedido almacena la cantidad de unidades adquiridas de una edición concreta y el precio unitario en el momento de la compra.

Este diseño permite mantener la coherencia histórica de los pedidos incluso si el precio de una edición cambia posteriormente.

3.10.7 Justificación del Modelo

El modelo entidad-relación ha sido diseñado con el objetivo de proporcionar una estructura de datos flexible, escalable y coherente con los requisitos funcionales del sistema.

Se ha optado por separar claramente las entidades relacionales con usuarios, catalogo, interacción social y sistema de compra para facilitar la organización lógica del sistema y simplificar su mantenimiento.

Asimismo, se han utilizado tablas intermedias para modelar correctamente las relaciones muchos a muchos, como en el caso de obras y autores o usuarios y eventos.

La separación entre obra y edición permite gestionar correctamente diferentes versiones de una misma obra publicadas por distintas editoriales, manteniendo el control del stock en el nivel adecuado.

Este modelo constituye la base para la implementación de la base de datos relacional del sistema y para la posterior definición de las entidades JPA utilizadas en el backend desarrollado con Spring Boot.

4. APÉNDICES

4.1 Formatos de Entrada y Salida de Datos

4.1.1 Formatos de Entrada

El sistema aceptará datos mediante formularios web estructurados.

Principales entradas del sistema:

- Registro de usuario
 - Nombre
 - Email
 - Contraseña
- Inicio de sesión
 - Email
 - Contraseña
- Publicación de comentario
 - Texto del comentario
 - Puntuación (1–5)
- Compra de productos
 - Identificador de edición
 - Cantidad seleccionada

Los datos enviados desde el cliente al servidor se transmitirán en formato JSON a través de la API REST.

Ejemplo estructural de entrada JSON:

```
{  
  "obraId": 12,  
  "cantidad": 2  
}
```

4.1.2 Formatos de Salida

El sistema devolverá respuestas en formato JSON.

Ejemplo de respuesta de obra:

```
{
  "id": 1,
  "titulo": "Naruto",
  "tipo": "MANGA",
  "valoracionMedia": 4.5,
  "stockDisponible": 25
}
```

Ejemplo de respuesta de pedido:

```
{
  "pedidoId": 45,
  "estado": "CONFIRMADO",
  "total": 39.99
}
```

4.2 Análisis de Costes (Estimación Académica)

Al tratarse de un proyecto académico, no existe un análisis económico real de mercado.

Sin embargo, se realiza una estimación teórica del coste de desarrollo:

- Tiempo estimado de desarrollo: 10–12 semanas.
- Horas aproximadas dedicadas: 200–300 horas.
- Coste estimado en entorno profesional (a 20€/hora):
Aproximadamente 4.000 – 6.000 €.

El uso de tecnologías open-source (Spring Boot, MySQL, Docker, Vue.js) reduce costes de licencia.

4.3 Restricciones acerca del Lenguaje de Programación

El sistema deberá desarrollarse bajo las siguientes restricciones tecnológicas:

- Backend obligatorio en Java.
- Uso del framework Spring Boot.
- Persistencia mediante JPA/Hibernate.
- Autenticación mediante JWT.
- Arquitectura basada en API REST.

- Base de datos relacional (H2 en desarrollo, MySQL en producción).

El frontend deberá desarrollarse utilizando:

- HTML5
 - CSS3
 - JavaScript
 - Framework Vue.js (segunda fase)
 - Thymeleaf (prototipo inicial)
-

4.4 Decisiones Arquitectónicas Relevantes

Se incluyen como referencia estructural las siguientes decisiones cerradas:

- El stock pertenece a la edición, no a la obra.
 - El sistema implementa comentarios jerárquicos.
 - La suscripción es única y de renovación automática.
 - El modelo de obras se basa en entidad base "Obra".
 - El sistema utiliza control de acceso basado en roles.
-

4.5 Planificación del Proyecto

El proyecto se desarrollará por hitos:

1. Documentación IEEE 830.
2. Desarrollo backend.
3. Desarrollo cliente.
4. Integración y despliegue.
5. Presentación final.